Average treatment effect for the ACTG data

Ezequiel

2025-01-08

```
# load libraries
library(hdbayes)
library(parallel)
library(matrixStats)
library(MCMCpack)
```

Distributions on the model space

Let $\mathcal{M}$ be the set of all possible model and

$$
\begin{aligned}
p(\beta^{(m)} \mid D_0^{(m)}) &= L(\beta^{(m)} \mid D_0^{(m)})\pi_0(\beta^{(m)} \mid d_0), \\
p_0(m) &= \int p(\beta^{(m)} \mid D_0^{(m)}) d\beta^{(m)}, \\
\beta^{(m)} &\sim N(\mathbf{0}, d_0 \times I_p).
\end{aligned}
$$

where $L(\beta^{(m)} \mid D_0^{(m)})$ is the likelihood of the historical data $D_0^{(m)}$ under model m, $\pi_0(\beta^{(m)} \mid d_0)$ is the prior distribution of the regression coefficients, and d_0 is the prior precision of the regression coefficients. We assume that the prior probability of each model is given by

$$
p(m) = \frac{p_0(m)}{\sum_{m \in \mathcal{M}} p_0(m)}.
$$

Now, we set the joint prior distribution for the regression coefficients and a_0 as

$$
\begin{aligned}
\pi(\beta^{(m)}, a_0 \mid D_0^{(m)}) &\propto \frac{L(\beta^{(m)} \mid D_0^{(m)})^{a_0}}{Z(a_0)} a_0^{\delta_0 - 1}(1 - a_0)^{\lambda_0 - 1}\pi_0(\beta^{(m)} \mid c_0), \\
Z(a_0) &= \int L(\beta^{(m)} | D_0^{(m)})^{a_0}\pi_0(\beta^{(m)}) d\beta^{(m)}
\end{aligned}
$$

and the posterior distribution of the model m as

$$
\begin{aligned}
p(m \mid D^{(m)}) &= \frac{p(D^{(m)} \mid m)p(m)}{\sum_{m \in \mathcal{M}} p(D^{(m)} \mid m)p(m)} = \frac{p(D^{(m)} \mid m)p_0(m)}{\sum_{m \in \mathcal{M}} p(D^{(m)} \mid m)p_0(m)}, \\
p(D^{(m)} \mid m) &= \int L(\beta^{(m)} \mid D^{(m)})\pi(\beta^{(m)}, a_0 \mid D_0^{(m)}) d\beta^{(m)} da_0.
\end{aligned}
$$

Estimation of the average treatment effect

The average treatment effect can be computed through the following steps:

1. Split the data into two groups: control ($D_0^{(m)}$) and treatment ($D_1^{(m)}$).
2. Compute the posterior distribution of each model m for the control and treatment groups, $p(m \mid D_0^{(m)})$ and $p(m \mid D_1^{(m)})$, for $m \in \mathcal{M}$.
3. Compute the posterior distribution of the regression coefficients for each model m in the control and treatment groups, $p(\beta^{(m)} \mid D_0^{(m)})$ and $p(\beta^{(m)} \mid D_1^{(m)})$, for $m \in \mathcal{M}$, and get $\mu_{am} \approx \sum_{i=1}^{n} w_i \text{logit}^{-1}(X_{mi}^{\top}\beta_a^{(m)})$, where n is the number of observations and $\beta_a^{(m)}$ is an the regression coefficients, for model m in group $a \in \{0,1\}$, $(w_1, \ldots, w_n) \sim \text{Dirichlet}(1, \ldots, 1)$, and X_{mi} is the matrix of covariates.
4. Get $\mu_a = \sum_{m \in \mathcal{M}} p(m \mid D_a^{(m)})\mu_{am}$, for $a \in \{0,1\}$.
5. Compute the average treatment effect as $\mu_1 - \mu_0$.

Computations

Posterior samples

To compute p_0, we use the glm.npp.lognc function with $a_0 = 1$. To compute $Z(a_0)$, we use the same function with different values of a_0. To compute the posterior for the regression coefficients, we use the glm.npp function. Lastly, to compute $p(D^{(m)} \mid m)$, we use the glm.logml.npp function.

We create auxiliary functions to compute posterior model probabilities:

```
# function to compute p0
logp0 <- function(formula,
                  ...) {
  hdbayes::glm.npp.lognc(
    formula = formula,
    ...
  )
}

# function to compute Z(a_0)
logncfun <- function(a0,
                     ...){
  hdbayes::glm.npp.lognc(
    a0 = a0,
    ...
  )
}

# function to compute the posterior for regression coefficients
post_beta <- function(formula,
                      c0,
                      data,
                      family,
                      delta0,
                      lambda0,
                      iter_warmup,
```

```
                      iter_sampling) {
  # compute Z(a_0) for each value of a_0
  a0.lognc <- lapply(a0_seq,
                     logncfun,
                     formula = formula,
                     family = family,
                     histdata = data[[2]],
                     iter_warmup = iter_warmup,
                     iter_sampling = iter_sampling,
                     chains = 4,
                     refresh = 0,
                     beta.sd = c0
  )
  a0.lognc <- data.frame(do.call(rbind, a0.lognc))
  # compute the posterior for the regression coefficients
  fit <- hdbayes::glm.npp(formula,
                          data = data,
                          family = family,
                          a0.lognc = a0.lognc$a0,
                          lognc = matrix(a0.lognc$lognc, ncol = 1),
                          a0.shape1 = delta0,
                          a0.shape2 = lambda0,
                          iter_warmup = iter_warmup,
                          iter_sampling = iter_sampling,
                          chains = 4,
                          refresh = 0,
                          beta.sd = c0
  )
}

# function to compute the posterior model probabilities
post_models <- function(logp0_models, logml_models) {
  logp0_models <- sapply(logp0_models, function(x) x[2])
  logml_models <- unlist(data.frame(do.call(rbind, logml_models))$logml)
  post_model <- exp(logml_models + logp0_models -
                      logSumExp(logml_models + logp0_models))
}
```

Lastly, we create a function to compute the results:

```
# function to compute the results
samples_models <- function(data,
                           outcome,
                           family,
                           a0_seq,
                           c0,
                           d0,
                           delta0,
                           lambda0,
                           iter_warmup,
                           iter_sampling,
                           num_cores = 10) {
  # get data
```

```r
current_data <- data[[1]]
hist_data <- data[[2]]

# create list of formulas
covariates <- colnames(current_data)[colnames(current_data) != outcome]
pset <- powerset(covariates)
formulas <- lapply(pset, create_formula, outcome = outcome)

# get covariates from models
covariates_models <- lapply(pset, get_covariates)

# define cluster
cl <- parallel::makeCluster(num_cores)
on.exit(parallel::stopCluster(cl))
parallel::clusterExport(cl, varlist = c('a0_seq',
                                        'family',
                                        'data',
                                        'c0',
                                        'd0',
                                        'delta0',
                                        'lambda0',
                                        'iter_warmup',
                                        'iter_sampling',
                                        'logncfun'),
                        envir = environment()
)
# compute p0
logp0_models <- parLapply(
  cl = cl,
  X = formulas,
  fun = logp0,
  family = family,
  histdata = hist_data,
  a0 = 1,
  iter_warmup = iter_warmup,
  iter_sampling = iter_sampling,
  chains = 4,
  refresh = 0,
  beta.sd = d0
)

# compute posterior for regression coefficients
post_betam  <- parLapply(
  cl = cl,
  X = formulas,
  fun = post_beta,
  c0 = c0,
  data = data,
  family = family,
  delta0 = delta0,
  lambda0 = lambda0,
  iter_warmup = iter_warmup,
  iter_sampling = iter_sampling
```

```
  )

  # compute logml for models
  logml_models <- parLapply(
    cl = cl,
    X = post_betam,
    fun = hdbayes::glm.logml.npp
  )

  # compute prior model probabilities
  prior_models <- exp(sapply(logp0_models, function(x) x[2]) -
                        logSumExp(sapply(logp0_models, function(x) x[2])))
  )

  # compute posterior model probabilities
  post_models <- post_models(logp0_models, logml_models)

  # create data frame with results
  df_post <- data.frame(model = unlist(covariates_models),
                        prior_model = prior_models,
                        ml = exp(unlist(data.frame(do.call(rbind, logml_models))$logml)),
                        post_model = post_models
  )

  # order data frame
  df_post_ord <- df_post[order(df_post$post_model, decreasing = TRUE),]
  rownames(df_post_ord) <- 1:nrow(df_post_ord)
  # return results
  return(list(logp0_models = logp0_models,
              post_betam = post_betam,
              logml_models = logml_models,
              df_post = df_post,
              df_post_ord = df_post_ord))
}
```

Average treatment effect

First, we normalize the data:

```
# normalize data
age_stats <- with(actg036,
                  c('mean' = mean(age), 'sd' = sd(age)))
cd4_stats <- with(actg036,
                  c('mean' = mean(cd4), 'sd' = sd(cd4)))
actg036$age <- ( actg036$age - age_stats['mean'] ) / age_stats['sd']
actg019$age <- ( actg019$age - age_stats['mean'] ) / age_stats['sd']
actg036$cd4 <- ( actg036$cd4 - cd4_stats['mean'] ) / cd4_stats['sd']
actg019$cd4 <- ( actg019$cd4 - cd4_stats['mean'] ) / cd4_stats['sd']
```

Then, we split the data:

```r
# define data
current_data <- actg036
hist_data <- actg019

# split data
current_data_ctrl <- current_data[current_data$treatment == 0,
                                  !(names(current_data) %in% c("treatment"))]
current_data_trt <- current_data[current_data$treatment == 1,
                                  !(names(current_data) %in% c("treatment"))]
```

Now, we set the parameters:

```r
beta_pars <- elicit_beta_mean_cv(m0 = 0.5, v0 = 0.005)
a0_seq <- seq(0, 1, length.out = 21)
data_ctrl <- list(current_data_ctrl, hist_data)
data_trt <- list(current_data_trt, hist_data)
family <- binomial(link = "logit")
c0 <- 3^0.5
d0 <- 0.5^0.5
delta0 <- beta_pars$a
lambda0 <- beta_pars$b
iter_warmup <- 1000
iter_sampling <- 2500
```

Finally, we run the model for each group:

```r
post_samples_ctrl <- samples_models(data = data_ctrl,
                                    outcome = "outcome",
                                    family = family,
                                    a0_seq = a0_seq,
                                    c0 = c0,
                                    d0 = d0,
                                    delta0 = delta0,
                                    lambda0 = lambda0,
                                    iter_warmup = iter_warmup,
                                    iter_sampling = iter_sampling,
                                    num_cores = 10)

post_samples_trt <- samples_models(data = data_trt,
                                    outcome = "outcome",
                                    family = family,
                                    a0_seq = a0_seq,
                                    c0 = c0,
                                    d0 = d0,
                                    delta0 = delta0,
                                    lambda0 = lambda0,
                                    iter_warmup = iter_warmup,
                                    iter_sampling = iter_sampling,
                                    num_cores = 10)
```

Now, we create auxiliary functions to compute the average treatment effect:

```r
# function to compute average effect of each model and group
mean_models_arm <- function(data, outcome, family, beta_post){
  # get covariates
  cov_data <- colnames(data)[colnames(data) != outcome]
  # get mean of beta posterior
  mean_post_betam <- lapply(beta_post, colMeans)
  # get product of beta posterior and covariates
  prod <- lapply(mean_post_betam,
                 function(post_samp) {
                   cov <- names(post_samp)[names(post_samp) %in% cov_data]
                   post_samp <- post_samp[c("(Intercept)", cov)]
                   cov_mat <- cbind(1, data.matrix(data[cov]))
                   beta_hat <- matrix(post_samp, ncol = 1)
                   cov_mat %*% beta_hat
                 }
  )
  # compute predictions
  if (family$link == "logit") {
    pred <- lapply(prod, plogis)
  } else if (family$link == "probit") {
    pred <- lapply(prod, pnorm)
  }
  # compute average effect
  sample_weights <- rdirichlet(1, rep(1, nrow(data)))
  mean_models <- lapply(pred,
                        function(x) {
                          sample_weights %*% x
                        }
  )
  # return average effect
  return(unlist(mean_models))
}

# function to compute average effect of each group
mean_arm <- function(post_models, mean_arm_models){
  sum(post_models * mean_arm_models)
}
```

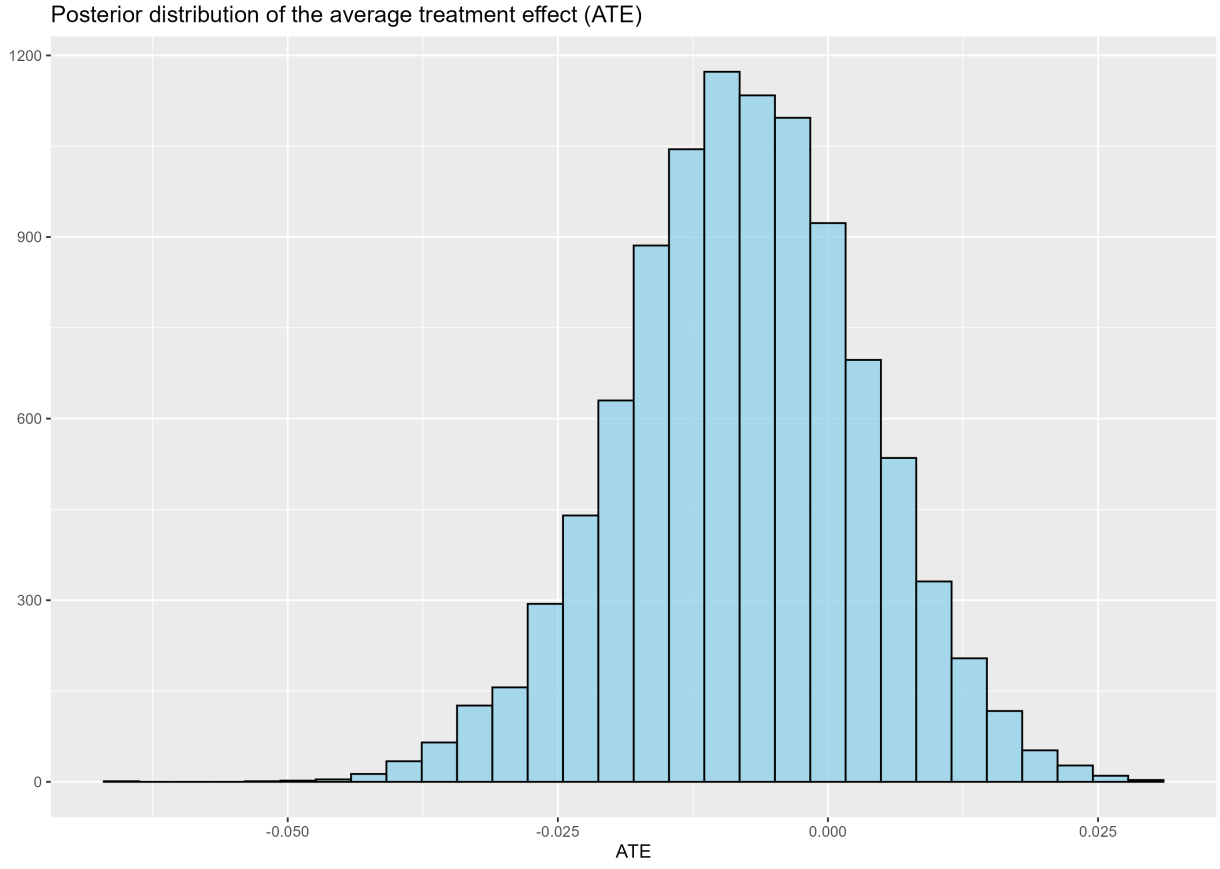
We can observe the results:

```r
post_samples_trt$df_post_ord
post_samples_ctrl$df_post_ord

#> post_samples_trt$df_post_ord
#>            model  prior_model           ml   post_model
#> 1            cd4 2.162312e-01 2.484787e-06 3.717036e-01
#> 2       age, cd4 4.273791e-01 1.150383e-06 3.401301e-01
#> 3 age, race, cd4 2.426728e-01 8.802853e-07 1.477862e-01
#> 4      race, cd4 1.135210e-01 1.787428e-06 1.403764e-01
#> 5            age 1.097845e-04 2.851294e-08 2.165570e-06
#> 6      age, race 6.291359e-05 2.168054e-08 9.436347e-07
#> 7           race 2.317249e-05 3.882745e-08 6.224450e-07
```

```
#> post_samples_ctrl$df_post_ord
#>            model  prior_model           ml   post_model
#> 1       age, cd4 4.280434e-01 5.416766e-10 4.723140e-01
#> 2 age, race, cd4 2.419794e-01 6.638414e-10 3.272243e-01
#> 3            cd4 2.165200e-01 2.666885e-10 1.176265e-01
#> 4      race, cd4 1.132612e-01 3.589752e-10 8.282250e-02
#> 5            age 1.099827e-04 3.149964e-11 7.057207e-06
#> 6      age, race 6.285405e-05 3.853906e-11 4.934433e-06
#> 7           race 2.323827e-05 1.686888e-11 7.985330e-07
```

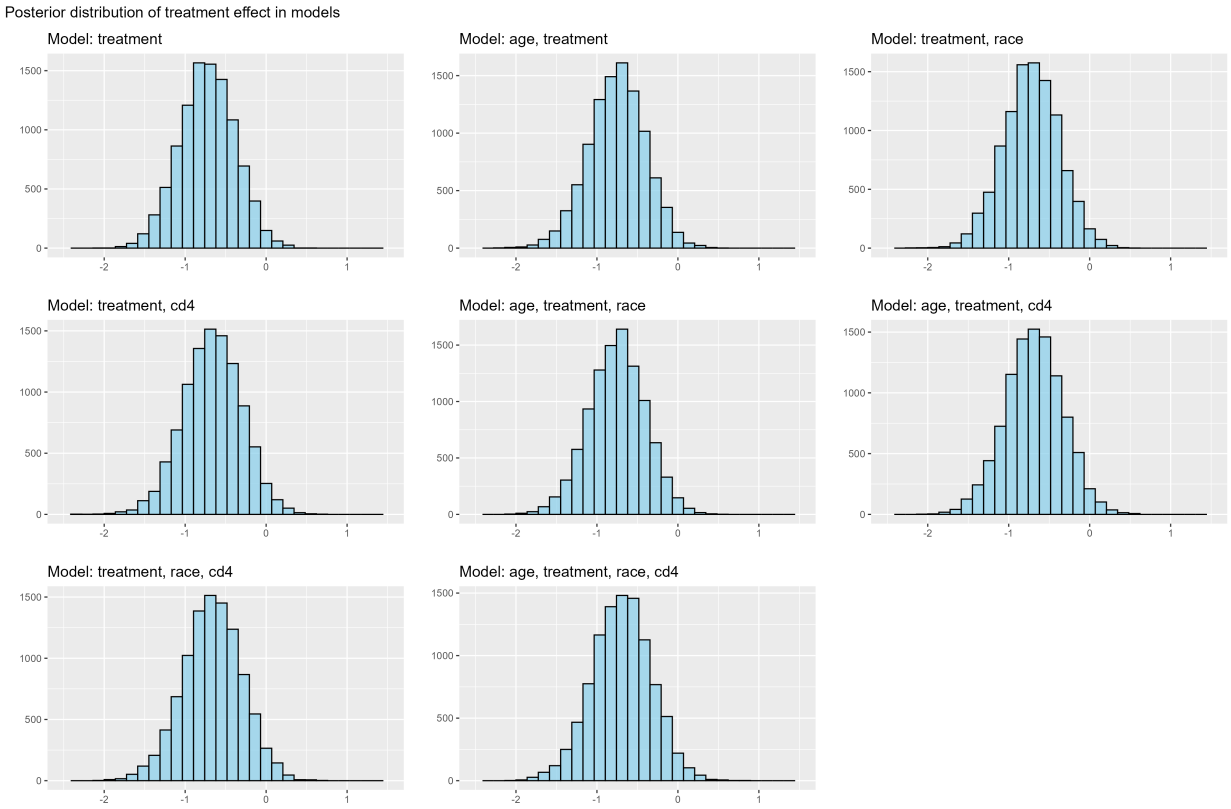
Finally, we compute the average treatment effect:

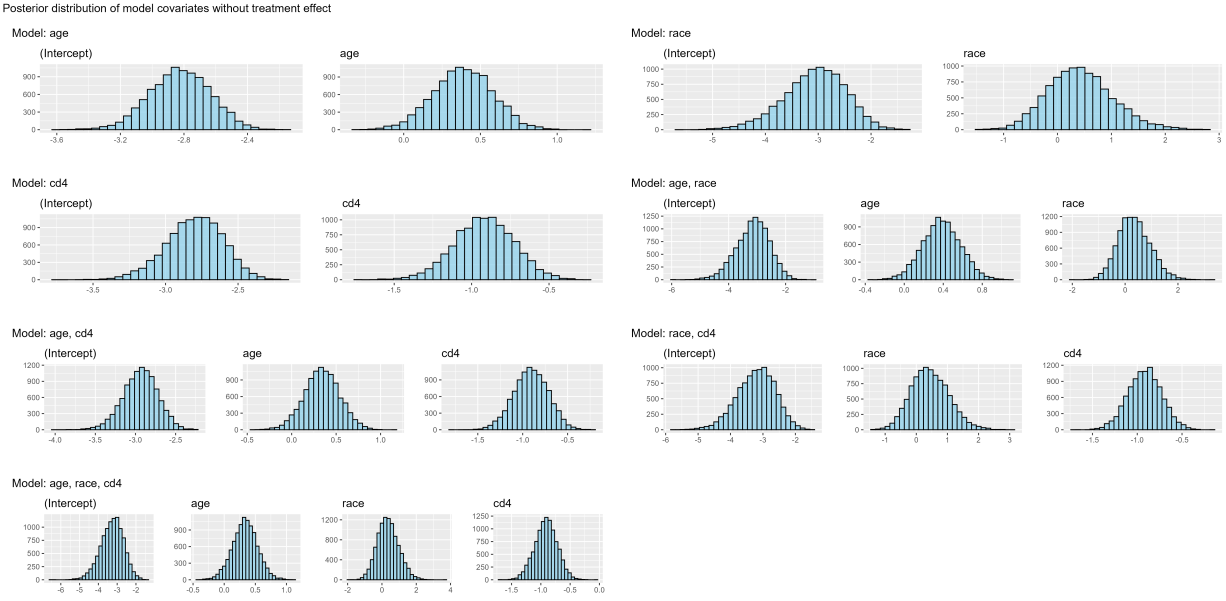
```
mean_models_ctrl <- mean_models_arm(current_data_ctrl, "outcome", family, post_samples_ctrl$post_betam)
mean_models_trt <- mean_models_arm(current_data_trt, "outcome", family, post_samples_trt$post_betam)
mean_ctrl <- mean_arm(post_samples_ctrl$df_post$post_model, mean_models_ctrl)
mean_trt <- mean_arm(post_samples_trt$df_post$post_model, mean_models_trt)

ate <- mean_trt - mean_ctrl
```



Intermediary results

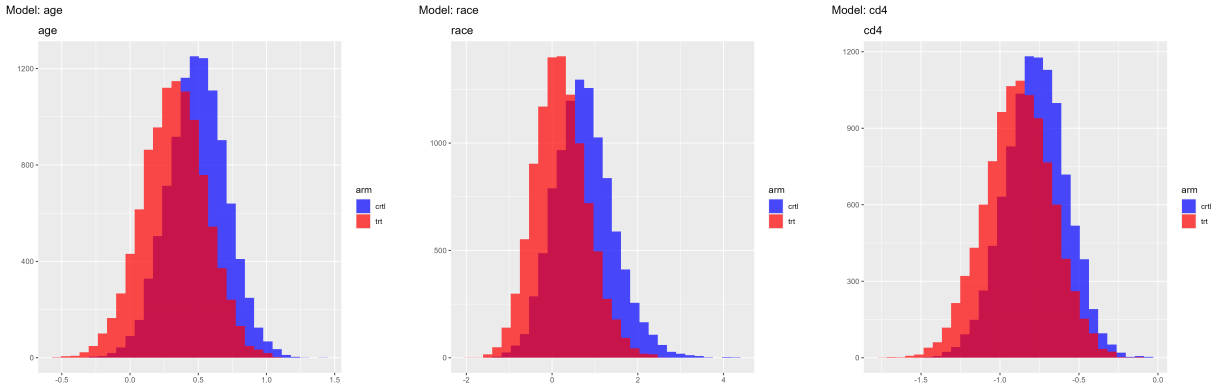
We can analyze the distribution of covariates in the complete data set:



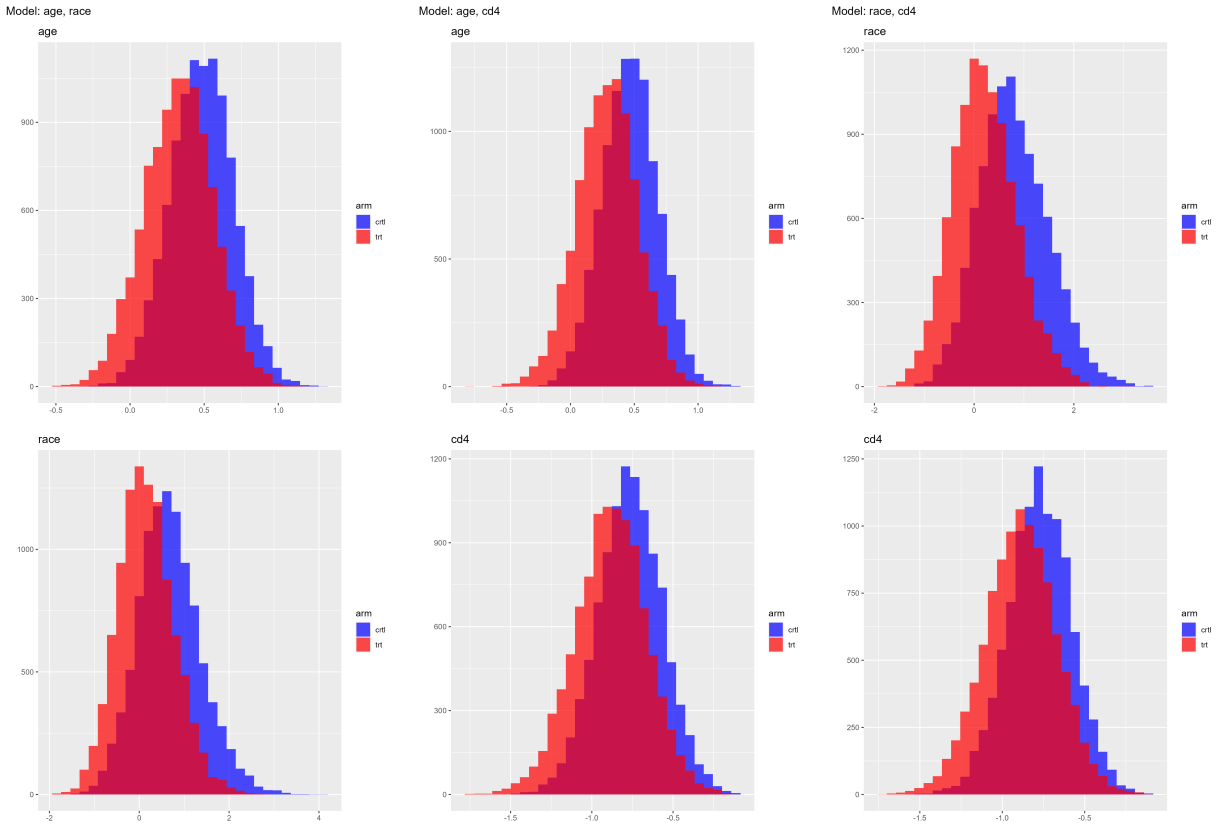


And the distribution of covariates by arm:

Posterior distribution of model covariates by arm - Part 1

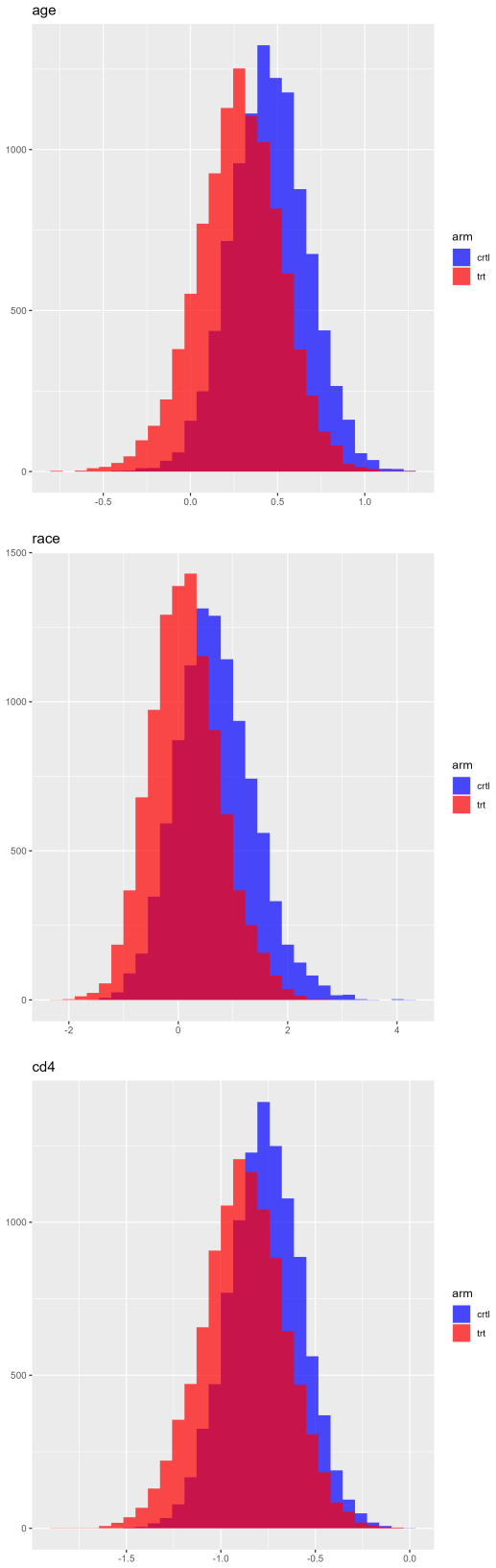


Posterior distribution of model covariates by arm - Part 2

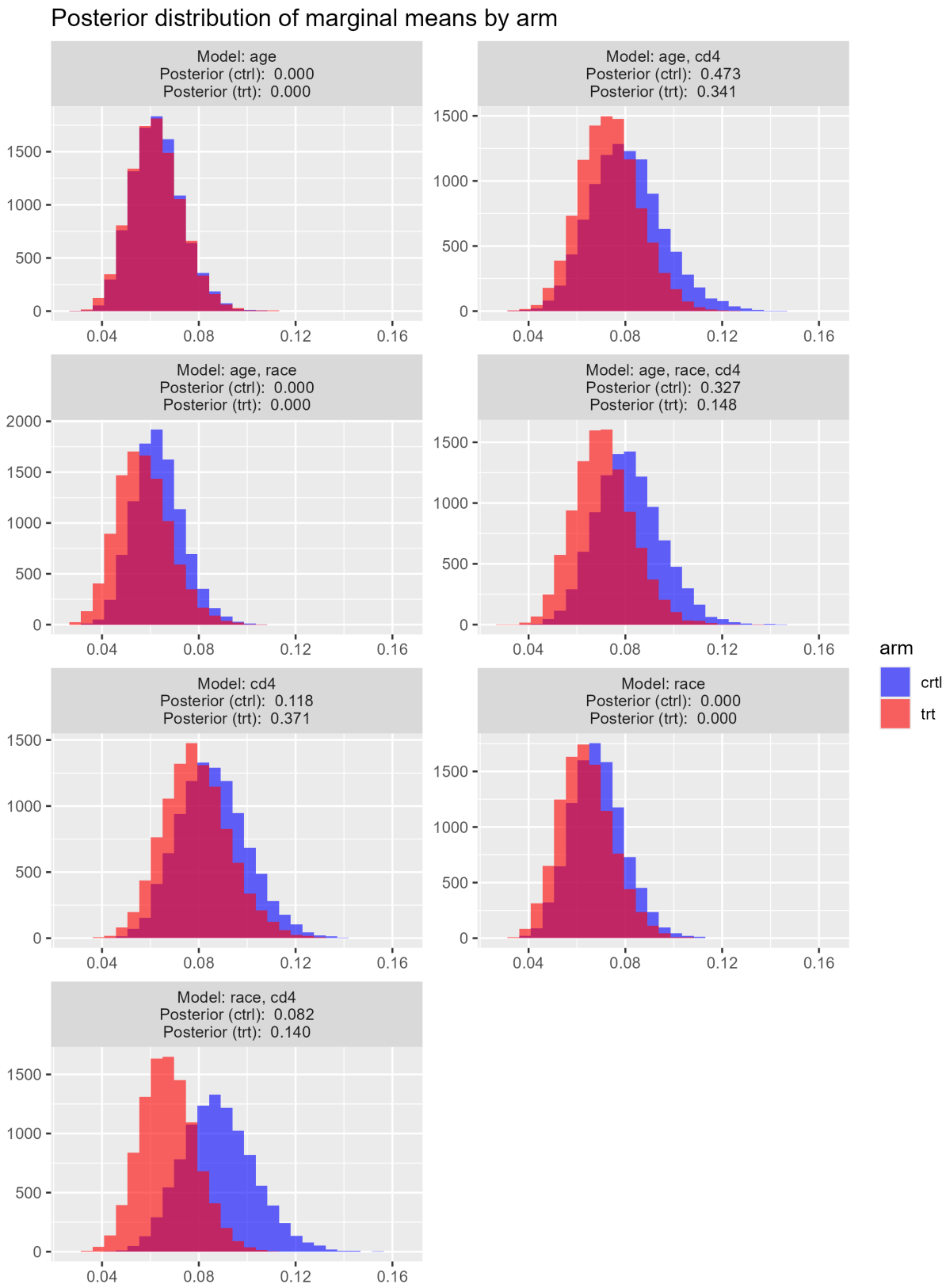


Posterior distribution of model covariates by arm - Part 3

Model: age, race, cd4



Lastly, we can observe the distribution of marginal means by arm:

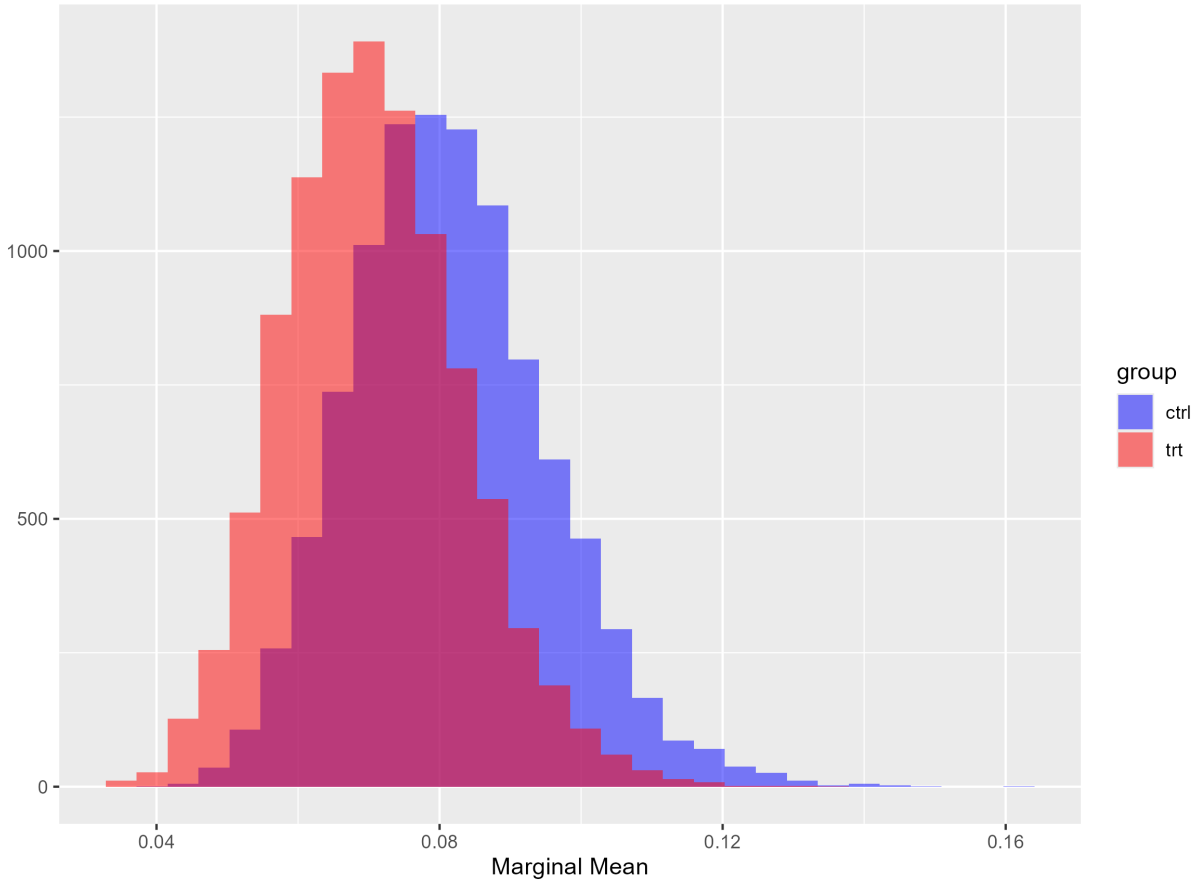


Another computation - Bayesian model averaging

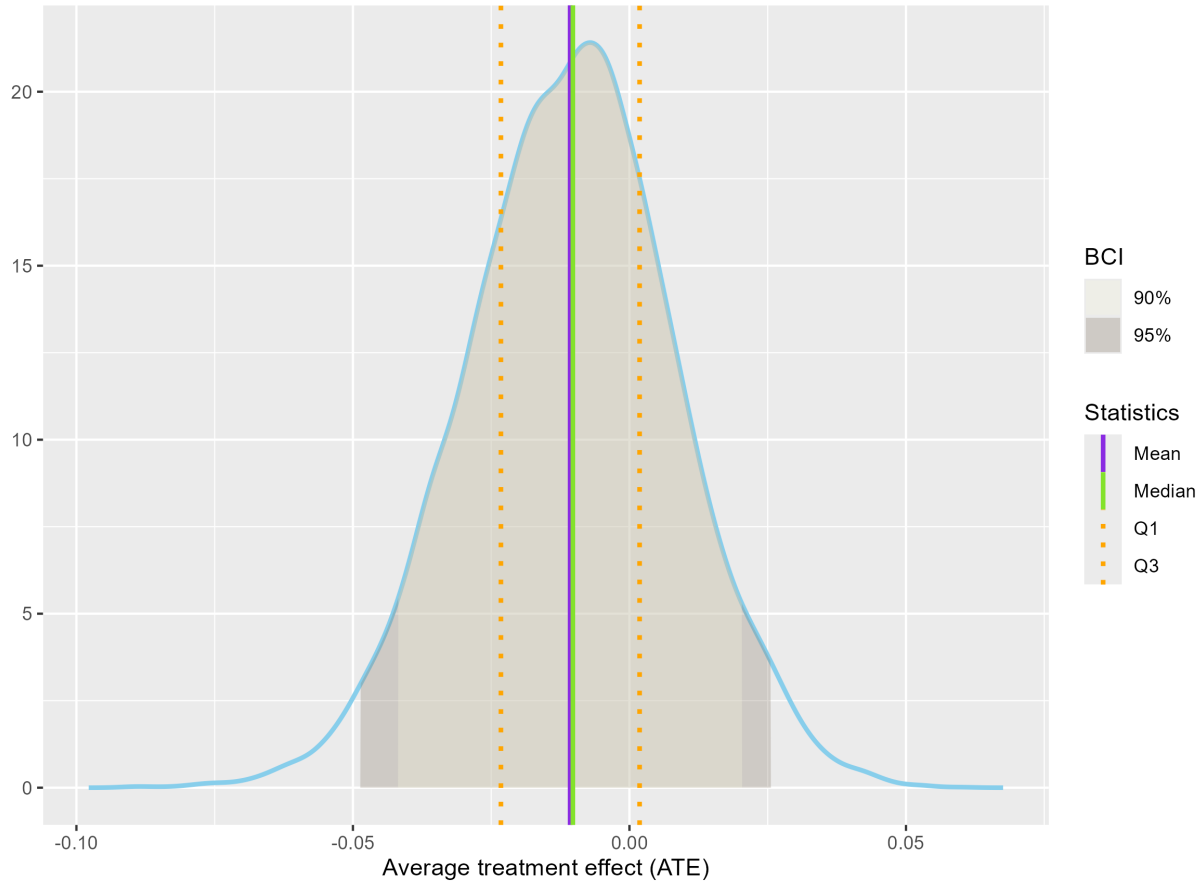
We can also compute the average treatment effect using Bayesian model averaging. The algorithm is as follows:

1. Compute the marginal means for each model and group.
2. For each group:
    1. Sample an index k from the posterior distribution of the models.
    2. Sample a draw from the posterior distribution of the marginal means for model k.
    3. Repeat the previous step N times.

From the samples, we can compute the marginal means for each group:



And the average treatment effect:



Finally, we can summarize the results:

statistic	value
Mean	-0.011
Median	-0.010
Q1	-0.023
Q3	0.002
90% BCI	[-0.042, 0.020]
95% BCI	[-0.049, 0.026]

Table 1: BMA summary (ATE)